

Lab 07 — Networking, Edge & Data Ingestion

Serve content through a caching edge, watch cache hits appear, ingest events into a stream, and see what the edge does when the origin fails.

Lab environment: Docker Desktop (free local simulation) — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: Design High-Performing Architectures

Overview

Local component	AWS equivalent
lab7-edge (NGINX proxy_cache, port 8087)	Amazon CloudFront edge location
lab7-origin	CloudFront origin (S3 bucket or ALB)
lab7-stream (Redis Streams)	Amazon Kinesis Data Streams

Step 1 — Get the lab source and start the lab

The repo lives in `~/Downloads/aws-saa`. The paths are absolute, so these commands work from any folder. `git clone` is needed only once: if the repo is already there it prints `already exists`, and the next line still takes you into this lab.

```
cd ~/Downloads
git clone https://github.com/mohanpothula/aws-saa.git
cd ~/Downloads/aws-saa/labs/lab-07-networking-edge-ingestion
docker compose up -d
docker compose ps
```

Step 2 — Edge endpoint

Task: Access content through the edge proxy.

```
curl -i http://localhost:8087
```

Expected result: `HTTP/1.1 200 OK` is returned.

Step 3 — Cache validation

Task: Request twice and inspect the cache header.

```
curl -I http://localhost:8087
curl -I http://localhost:8087
```

Expected result: `X-Cache: MISS` on the first request, `X-Cache: HIT` on the second.

Step 4 — Create stream events

Task: Add three events to the stream.

```
docker exec lab7-stream redis-cli XADD events '*' type order id 1001
docker exec lab7-stream redis-cli XADD events '*' type order id 1002
docker exec lab7-stream redis-cli XADD events '*' type payment id 1001
```

Expected result: Three entry IDs are returned (e.g. `1757574123456-0`).

Step 5 — Consume events

Task: Read the stream.

```
docker exec lab7-stream redis-cli XRange events - +
```

Expected result: All three events are listed.

Step 6 — Edge failure test

Task: Stop the origin and request cached content.

```
docker compose stop origin
curl http://localhost:8087
```

Expected result: Content cached in Step 3 is still served (it is valid for 10 minutes). Discuss how a CDN behaves when its origin is down.

Verification

Check	Expected
Step 2 — Edge endpoint	<code>HTTP/1.1 200 OK</code> is returned
Step 3 — Cache validation	<code>X-Cache: MISS</code> on the first request, <code>X-Cache: HIT</code> on the second
Step 4 — Create stream events	Three entry IDs are returned (e.g. <code>1757574123456-0</code>)
Step 5 — Consume events	All three events are listed
Step 6 — Edge failure test	Content cached in Step 3 is still served (it is valid for 10 minutes). Discuss how a CDN behaves when its origin is down

Completion check

- [] Every validation result demonstrated to the trainer
- [] You can say which AWS service each local component represents
- [] You can explain why the edge kept serving with the origin down

Cleanup

```
docker compose down -v
```

`-v` also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code:
<https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-07-networking-edge-ingestion>
- Original student guide: [courseware/AWS_SAA_Lab7_Student_Guide.docx](#)
- [Amazon CloudFront Developer Guide](#)
- [CloudFront caching and availability](#)
- [Amazon Kinesis Data Streams](#)
- [NGINX proxy module \(proxy_pass, proxy_cache\)](#)
- [Redis Streams](#)